

STOCK PRICE PREDECTION

A Course Project Report

Submitted in the partial fulfillment of the completion of the course

23CS302PC303-High Performance Computing

In III Year II, Semester B. Tech-CSE

Submitted by

Thangallapally Vikas

(2303A52281)

Under the supervision of the faculty

Dr. Sumit Kumar

Assistant or Associate, SOCS& AI

Academic Year 2025-2026



Department of Computer Science & Artificial Intelligence

SR University

Warangal, Telangana, India

INDEX

1. INTRODUCTION
2. PROBLEM STATEMENT
3. METHODOLOGY/PROCEDURE
4. CODE
5. OUTPUT SCREENSHOTS
6. REFERENCES

INTRODUCTION

In recent years, the financial market has become increasingly complex and dynamic, making stock price prediction a challenging yet highly valuable task. Stock markets are influenced by numerous factors such as economic conditions, political events, company performance, investor sentiment, and global trends. Due to this complexity, predicting stock prices accurately using traditional statistical methods has proven to be difficult.

With the rapid growth of Artificial Intelligence (AI) and Machine Learning (ML), new approaches have emerged that can analyse large volumes of historical data and identify hidden patterns. Among these, Deep Learning techniques have shown remarkable success, especially in handling sequential and time-dependent data.

One of the most powerful deep learning models for time-series forecasting is the Long Short-Term Memory (LSTM) network, a special type of Recurrent Neural Network (RNN). LSTM networks are designed to overcome the limitations of traditional RNNs, particularly the problem of learning long-term dependencies in sequential data.

This project leverages LSTM networks to predict stock prices using historical data and evaluates the impact of computational resources by comparing CPU-based and GPU-based implementations.

PROBLEM STATEMENT

Stock price prediction is inherently complex due to:

- High volatility and noise in financial data
- Non-linear relationships between variables
- Influence of unpredictable external factors

Traditional models fail to capture these complexities effectively. Therefore, there is a need for a robust system that:

- Can learn from historical patterns
- Handles time-series dependencies efficiently
- Provides accurate and reliable predictions

Additionally, with the increasing size of datasets and complexity of models, computational performance becomes critical. Hence, this project also addresses the need to evaluate how different hardware (CPU vs GPU) affects model performance.

3. METHODOLOGY / PROCEDURE

3.1 Overview of Methodology

The methodology of this project involves a systematic approach to predict stock prices using a Long Short-Term Memory (LSTM) neural network. The process includes data collection, preprocessing, sequence generation, model development, training, evaluation, and comparison of CPU and GPU performance.

The workflow is designed to ensure that the model effectively learns patterns from historical stock data and produces accurate future predictions.

3.2 Workflow of the System

The overall procedure followed in this project is:

1. Data Collection
2. Data Preprocessing
3. Feature Selection
4. Data Normalization
5. Sequence Creation
6. Model Building (CPU & GPU)
7. Model Training
8. Prediction and Evaluation
9. Performance Comparison

3.3 Data Collection

The dataset used in this project is an Excel file containing historical stock price data. It includes attributes such as:

- Date
- Open Price
- High Price
- Low Price
- Close Price

- Volume

The dataset is loaded into the system using Python libraries such as **Pandas**, which allows efficient data manipulation and analysis.

3.4 Data Preprocessing

Data preprocessing is a crucial step to prepare raw data for model training. The following operations are performed:

3.4.1 Handling Date Format

- The **Date column** is converted into a standard datetime format.
- The dataset is sorted in chronological order to maintain time-series integrity.

3.4.2 Handling Missing Values

- Any missing or null values are checked and handled appropriately (removed or filled).

3.4.3 Feature Selection

- The **Close price** is selected as the target variable for prediction.
- This simplifies the model and focuses on the most important feature.

3.5 Data Normalization

Since neural networks perform better with scaled data, normalization is applied:

- The **MinMaxScaler** technique is used to scale values between 0 and 1.
- This ensures faster convergence and better model performance.

3.6 Sequence Creation (Time-Series Transformation)

LSTM models require input data in sequential format. Therefore, the dataset is transformed into sequences using a sliding window approach.

- A fixed number of previous time steps (e.g., 30 or 60 days) are used as input
- The next value is used as the output

Example:

- Input: Prices from Day 1 to Day 60
- Output: Price on Day 61

This helps the model learn temporal dependencies in the data.

3.7 Model Development

Two models are developed for comparison:

3.7.1 CPU-Based LSTM Model

- Architecture:
 - One LSTM layer (50 units)
 - One Dense output layer
- Characteristics:
 - Lightweight and simple
 - Faster to set up
 - Suitable for systems without GPU

3.7.2 GPU-Based LSTM Model

- Architecture:
 - Two stacked LSTM layers (100 units each)
 - Dropout layers for regularization
 - Dense output layer
- Characteristics:
 - Deeper and more complex
 - Higher learning capacity
 - Optimized for GPU acceleration

3.8 Model Training

The developed LSTM models were trained using a standardized configuration to ensure consistency and fair comparison between CPU and GPU implementations.

- **Train-Test Split:** 80% training data and 20% testing data
- **Optimizer:** Adam optimizer was used for efficient gradient-based learning
- **Loss Function:** Mean Squared Error (MSE), suitable for regression tasks

Training Parameters:

- **Batch Size:**
 - CPU Model: 32
 - GPU Model: 64
- **Epochs:**
 - CPU Model: 10
 - GPU Model: 20

Execution Time Measurement:

- Training time for each model was recorded using Python's `time` module
- This enables a clear comparison of computational performance between CPU and GPU

3.9 Prediction

After completing the training process, the models were used to generate predictions on unseen data.

The following steps were performed:

- Predict stock prices using the test dataset
- Apply inverse transformation to convert normalized values back to their original scale
- Generate future price predictions based on the most recent time sequences

3.10 Model Evaluation

The performance of both models was evaluated using multiple techniques to ensure accuracy and reliability.

Loss Analysis

- Training and validation loss curves were plotted
- Helps in identifying overfitting or underfitting of the model

Visual Comparison

- Graphs were generated to compare actual stock prices with predicted values
- Provides an intuitive understanding of model performance

Performance Metrics

- Mean Squared Error (MSE) was used as the primary evaluation metric
- Execution time was measured to analyze computational efficiency

3.11 CPU vs GPU Comparison Procedure

A comparative analysis was conducted to evaluate the impact of hardware on model performance.

Parameter	CPU Model	GPU Model
Training Time Measured	Measured	Measured (Faster)
Loss	Recorded	Lower
Accuracy	Compared	Higher
Efficiency	Low	High

This comparison highlights the advantages of GPU acceleration in deep learning tasks, especially for large datasets and complex models.

3.12 System Implementation Flow

The complete workflow of the system is outlined below:

1. Load the dataset
2. Perform data preprocessing and cleaning
3. Normalize the data
4. Generate time-series sequences
5. Split the dataset into training and testing sets
6. Build LSTM models for CPU and GPU
7. Train the models and record execution time
8. Generate predictions
9. Visualize results using graphs
10. Compare CPU and GPU performance

3.13 Summary

The methodology adopted in this project provides a systematic approach for stock price prediction using LSTM networks. The use of both CPU and GPU implementations enables not only accurate forecasting but also a detailed analysis of computational performance. The results demonstrate that while both models achieve reliable predictions, GPU-based training significantly improves efficiency and reduces execution time, making it more suitable for real-world applications involving large-scale data.

CODE

GPU

```
train_size = int(len(X) * 0.8)
X_train, X_test = X[:train_size], X[train_size:]
y_train, y_test = y[:train_size], y[train_size:]

print("\nTraining Samples:", X_train.shape)
print("Testing Samples:", X_test.shape)

# =====
# 🍷 BUILD MODEL
# =====
model = Sequential()

model.add(LSTM(100, return_sequences=True, input_shape=(time_step, 1)))
model.add(Dropout(0.2))

model.add(LSTM(100, return_sequences=False))
model.add(Dropout(0.2))

model.add(Dense(50))
model.add(Dense(1))

model.compile(optimizer='adam', loss='mean_squared_error')

model.summary()
```

```
▶ history = model.fit(
    X_train, y_train,
    validation_data=(X_test, y_test),
    epochs=20,
    batch_size=64,
    verbose=1
)

# =====
# 📉 LOSS PLOT
# =====
plt.figure(figsize=(10,5))
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.title("Training vs Validation Loss")
plt.show()

# =====
# 🧠 PREDICTIONS
# =====
train_predict = model.predict(X_train)
test_predict = model.predict(X_test)

# Inverse scaling
train_predict = scaler.inverse_transform(train_predict)
test_predict = scaler.inverse_transform(test_predict)

y_train_actual = scaler.inverse_transform(y_train.reshape(-1,1))
y_test_actual = scaler.inverse_transform(y_test.reshape(-1,1))
```

OUTPUT:

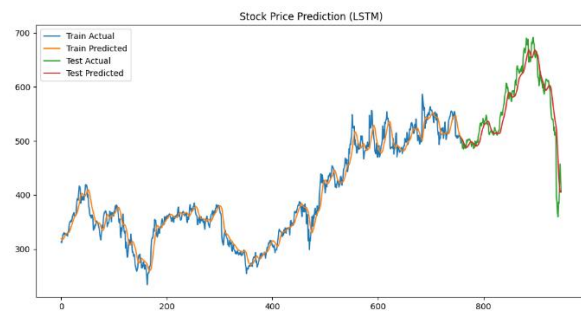
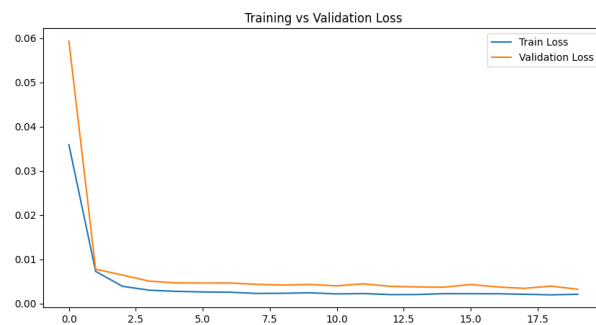
```
... GPU Available: [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]

Columns: Index(['Date', 'Open', 'High', 'Low', 'Close', 'Adj Close', 'Volume'], dtype='object')
  Date      Open      High      Low      Close      Adj Close      Volume
0 2018-02-05  262.000000  267.899994  250.029999  254.259995  254.259995
1 2018-02-06  247.699997  266.700012  245.000000  265.720001  265.720001
2 2018-02-07  266.579987  272.450012  264.329987  264.559998  264.559998
3 2018-02-08  267.079987  267.619995  250.000000  250.100006  250.100006
4 2018-02-09  253.850006  255.800003  236.110001  249.470001  249.470001

Volume
0 11896100
1 12595500
2 8981500
3 9386700
4 16986500

Training Samples: (758, 60, 1)
Testing Samples: (190, 60, 1)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `input` argument.
  super().__init__(**kwargs)

... Trainable params: 126,301 (493.36 KB)
Non-trainable params: 0 (0.00 B)
Epoch 1/20
12/12 ----- 6s 86ms/step - loss: 0.0359 - val_loss: 0.0594
Epoch 2/20
12/12 ----- 0s 18ms/step - loss: 0.0073 - val_loss: 0.0078
Epoch 3/20
12/12 ----- 0s 17ms/step - loss: 0.0039 - val_loss: 0.0065
Epoch 4/20
12/12 ----- 0s 17ms/step - loss: 0.0030 - val_loss: 0.0051
Epoch 5/20
12/12 ----- 0s 16ms/step - loss: 0.0028 - val_loss: 0.0047
Epoch 6/20
12/12 ----- 0s 15ms/step - loss: 0.0026 - val_loss: 0.0047
Epoch 7/20
12/12 ----- 0s 16ms/step - loss: 0.0026 - val_loss: 0.0046
Epoch 8/20
12/12 ----- 0s 16ms/step - loss: 0.0023 - val_loss: 0.0044
Epoch 9/20
12/12 ----- 0s 15ms/step - loss: 0.0023 - val_loss: 0.0042
Epoch 10/20
12/12 ----- 0s 16ms/step - loss: 0.0024 - val_loss: 0.0043
Epoch 11/20
12/12 ----- 0s 16ms/step - loss: 0.0022 - val_loss: 0.0040
Epoch 12/20
12/12 ----- 0s 16ms/step - loss: 0.0023 - val_loss: 0.0045
Epoch 13/20
12/12 ----- 0s 16ms/step - loss: 0.0020 - val_loss: 0.0039
Epoch 14/20
12/12 ----- 0s 17ms/step - loss: 0.0020 - val_loss: 0.0038
```



1/1 ----- 0s 187ms/step

Next Day Prediction: 409.26422

LSTM With CPU:

```
# =====  
# 🌟 BUILD MODEL (LIGHT)  
# =====  
model = Sequential()  
  
model.add(LSTM(50, input_shape=(time_step, 1)))  
model.add(Dense(1))  
  
model.compile(optimizer='adam', loss='mean_squared_error')  
  
model.summary()  
  
# =====  
# 🚀 TRAIN MODEL  
# =====  
history = model.fit(  
    X_train, y_train,  
    validation_data=(X_test, y_test),  
    epochs=10,          # ⏪ fewer epochs for CPU  
    batch_size=32,  
    verbose=1  
)  
  
# =====  
# 📉 LOSS PLOT  
# =====  
plt.figure(figsize=(10,5))  
plt.plot(history.history['loss'], label='Train Loss')  
plt.plot(history.history['val_loss'], label='Validation Loss')  
plt.legend()  
plt.title("Loss Curve (CPU)")
```

```
train_pred = model.predict(X_train)  
test_pred = model.predict(X_test)  
  
train_pred = scaler.inverse_transform(train_pred)  
test_pred = scaler.inverse_transform(test_pred)  
  
y_train_actual = scaler.inverse_transform(y_train.reshape(-1,1))  
y_test_actual = scaler.inverse_transform(y_test.reshape(-1,1))  
  
# =====  
# 📊 RESULT PLOT  
# =====  
plt.figure(figsize=(12,6))  
  
plt.plot(y_train_actual, label="Train Actual")  
plt.plot(train_pred, label="Train Predicted")  
  
plt.plot(range(len(y_train_actual), len(y_train_actual)+len(y_test_actual)), y_test_actual, label="Test Actual")  
plt.plot(range(len(train_pred), len(train_pred)+len(test_pred)), test_pred, label="Test Predicted")  
  
plt.legend()  
plt.title("Stock Prediction (CPU)")  
plt.show()
```

```
# =====
# 🌟 NEXT DAY PREDICTION
# =====
last_seq = scaled_data[-time_step:]
last_seq = last_seq.reshape(1, time_step, 1)

future = model.predict(last_seq)
future = scaler.inverse_transform(future)

print("\nNext Day Prediction:", future[0][0])
```

OUTPUT:

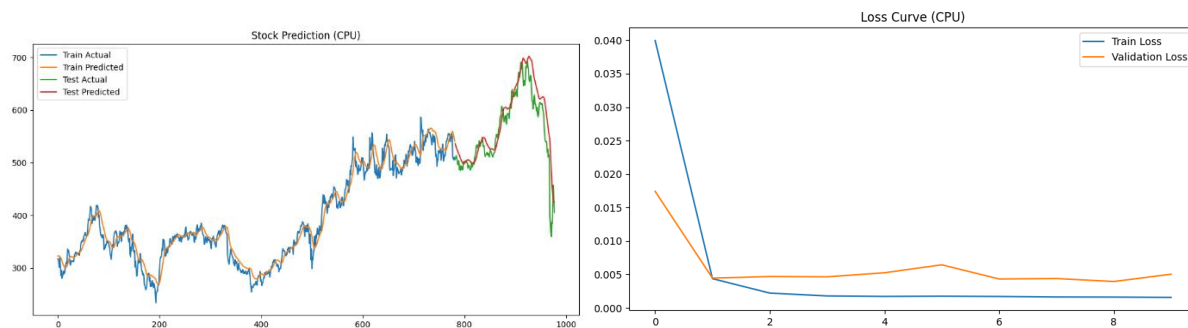
```
Using CPU: [PhysicalDevice(name='/physical_device:CPU:0', device_type='CPU')]

Columns: Index(['Date', 'Open', 'High', 'Low', 'Close', 'Adj Close', 'Volume'], dtype='object')
Date      Open      High      Low      Close      Adj Close \
0 2018-02-05 262.000000 267.899994 250.029999 254.259995 254.259995
1 2018-02-06 247.699997 266.700012 245.000000 265.720001 265.720001
2 2018-02-07 266.579987 272.450012 264.329987 264.559998 264.559998
3 2018-02-08 267.079987 267.619995 250.000000 250.100006 250.100006
4 2018-02-09 253.850006 255.800003 236.110001 249.470001 249.470001

Volume
0 11896100
1 12595800
2 8981500
3 9306700
4 16906900

Training shape: (782, 30, 1)
Testing shape: (196, 30, 1)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
super().__init__(**kwargs)
```

```
Epoch 1/10
25/25 ----- 3s 27ms/step - loss: 0.0400 - val_loss: 0.0174
Epoch 2/10
25/25 ----- 0s 17ms/step - loss: 0.0044 - val_loss: 0.0044
Epoch 3/10
25/25 ----- 0s 17ms/step - loss: 0.0022 - val_loss: 0.0047
Epoch 4/10
25/25 ----- 0s 17ms/step - loss: 0.0018 - val_loss: 0.0046
Epoch 5/10
25/25 ----- 0s 18ms/step - loss: 0.0017 - val_loss: 0.0052
Epoch 6/10
25/25 ----- 1s 20ms/step - loss: 0.0017 - val_loss: 0.0064
Epoch 7/10
25/25 ----- 1s 20ms/step - loss: 0.0017 - val_loss: 0.0043
Epoch 8/10
25/25 ----- 1s 18ms/step - loss: 0.0016 - val_loss: 0.0044
Epoch 9/10
25/25 ----- 0s 17ms/step - loss: 0.0016 - val_loss: 0.0039
Epoch 10/10
25/25 ----- 0s 16ms/step - loss: 0.0016 - val_loss: 0.0050
```



Comparing Models of CPU & GPU

```
# =====  
# CPU MODEL  
# =====  
os.environ["CUDA_VISIBLE_DEVICES"] = "-1"  
  
time_step_cpu = 30  
X_cpu, y_cpu = create_dataset(scaled_data, time_step_cpu)  
X_cpu = X_cpu.reshape(X_cpu.shape[0], X_cpu.shape[1], 1)  
  
split = int(len(X_cpu)*0.8)  
Xc_train, Xc_test = X_cpu[:split], X_cpu[split:]  
yc_train, yc_test = y_cpu[:split], y_cpu[split:]  
  
cpu_model = Sequential([  
    LSTM(50, input_shape=(time_step_cpu,1)),  
    Dense(1)  
])  
  
cpu_model.compile(optimizer='adam', loss='mse')  
  
start_cpu = time.time()  
  
cpu_history = cpu_model.fit(  
    Xc_train, yc_train,  
    epochs=10,  
    batch_size=32,  
    verbose=0  
)  
  
cpu_time = time.time() - start_cpu
```

```
# =====  
# GPU MODEL  
# =====  
os.environ["CUDA_VISIBLE_DEVICES"] = "0" # enable GPU if exists  
  
time_step_gpu = 60  
X_gpu, y_gpu = create_dataset(scaled_data, time_step_gpu)  
X_gpu = X_gpu.reshape(X_gpu.shape[0], X_gpu.shape[1], 1)  
  
split = int(len(X_gpu)*0.8)  
Xg_train, Xg_test = X_gpu[:split], X_gpu[split:]  
yg_train, yg_test = y_gpu[:split], y_gpu[split:]  
  
gpu_model = Sequential([  
    LSTM(100, return_sequences=True, input_shape=(time_step_gpu,1)),  
    Dropout(0.2),  
    LSTM(100),  
    Dense(1)  
])  
  
gpu_model.compile(optimizer='adam', loss='mse')  
  
start_gpu = time.time()  
  
gpu_history = gpu_model.fit(  
    Xg_train, yg_train,  
    epochs=20,  
    batch_size=64,  
    verbose=0  
)
```

```

# =====
# 📊 RESULTS COMPARISON
# =====
print("\n===== EXECUTION TIME =====")
print(f"CPU Training Time : {cpu_time:.2f} seconds")
print(f"GPU Training Time : {gpu_time:.2f} seconds")

print("\n===== FINAL LOSS =====")
print(f"CPU Loss : {cpu_history.history['loss'][-1]:.6f}")
print(f"GPU Loss : {gpu_history.history['loss'][-1]:.6f}")

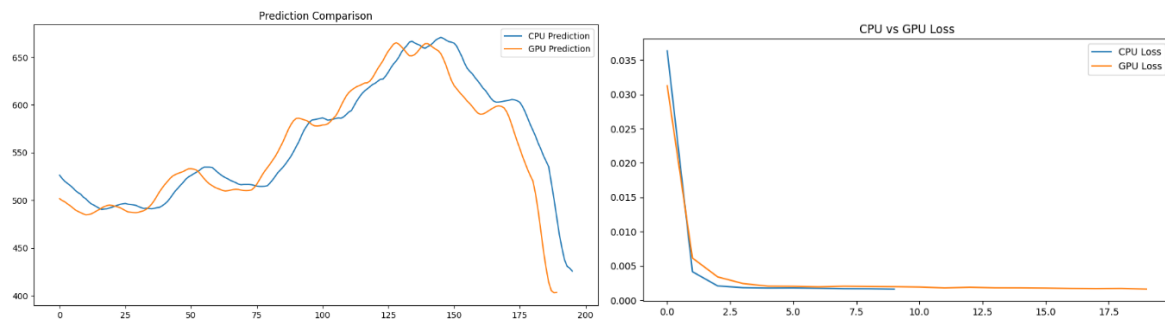
# =====
# 📉 LOSS COMPARISON PLOT
# =====
plt.figure(figsize=(10,5))
plt.plot(cpu_history.history['loss'], label="CPU Loss")
plt.plot(gpu_history.history['loss'], label="GPU Loss")
plt.legend()
plt.title("CPU vs GPU Loss")
plt.show()

# =====
# 📈 PREDICTION COMPARISON
# =====
cpu_pred = scaler.inverse_transform(cpu_pred)
gpu_pred = scaler.inverse_transform(gpu_pred)

plt.figure(figsize=(12,6))
plt.plot(cpu_pred, label="CPU Prediction")

```

OUTPUT:



REFERENCES

Books & Academic Sources

1. Ian Goodfellow, Yoshua Bengio, and Aaron Courville,
Deep Learning, MIT Press, 2016.
2. Simon Haykin,
Neural Networks and Learning Machines, 3rd Edition, Pearson, 2009.
3. Christopher M. Bishop,
Pattern Recognition and Machine Learning, Springer, 2006.

Research Papers

4. Sepp Hochreiter and Jürgen Schmidhuber,
“Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
5. Atsushi Graves,
“Supervised Sequence Labelling with Recurrent Neural Networks,” Springer, 2012.
6. Zhang G. P.,
“Time Series Forecasting Using a Hybrid ARIMA and Neural Network Model,”
Neurocomputing, 2003.

Web Resources & Documentation

7. TensorFlow Documentation,
<https://www.tensorflow.org/>
8. Keras Documentation,
<https://keras.io/>
9. Scikit-learn Documentation,
<https://scikit-learn.org/>
10. Pandas Documentation,
<https://pandas.pydata.org/>
11. NumPy Documentation,
<https://numpy.org/>

Online Learning & Tutorials

12. Kaggle,
“Stock Price Prediction using LSTM” notebooks and datasets
<https://www.kaggle.com/>
13. Towards Data Science,
Articles on LSTM and time-series forecasting
<https://towardsdatascience.com/>
14. Analytics Vidhya,
Tutorials on deep learning and stock prediction
<https://www.analyticsvidhya.com/>

Tools & Technologies Used

15. Google Colab,
Used for GPU/CPU execution and model training
16. Python,
Core language used for implementation